Optimization and matching solutions

1

Take $s\to a:2, s\to b:1, a\to b:-2$. Dijkstra finalizes b at one before a at two. Relaxing from a would produce true distance zero, but standard finalization no longer updates b, leaving the wrong answer. No cycle exists. This disproves removing the theorem's nonnegative-weight premise, not a claim that every negative-edge graph must fail.

2

Sorted edges are AB:1, BC:2, CD:3, AC:4, BD:5. Kruskal takes AB, BC, CD, total six. Cut {A} has AB, AC with AB lightest. The final cut {A, B, C}|{D} has both CD:3 and BD:5. CD is not the only crossing edge, but is uniquely lightest and must appear in every MST. The cut respects accepted AB, BC, so adding CD safely extends them; the result already is a spanning tree.

3

Let L_1, L_2 neighbor only R_1 and L_3 only R_2, leaving R_3 isolated. For $S=\{L_1,L_2\}$, $N(S)=\{R_1\}$, so $1<2$ violates Hall. Matching $\{L_1R_1, L_3R_2\}$ has size two and is maximum by that deficiency. Replacing L_1 by L_2 gives another maximum.

4

Each $s\to L$ and $R\to t$ arc has capacity one. Conservation bounds total flow through each left and right vertex by one. In an integral maximum flow, all arc flows are zero or one, so at most one used $L\to R$ edge touches each endpoint. They form a matching. Integer capacities guarantee existence of an integral optimum by integer augmentations from zero; they do not make every feasible flow integral.

5

Send two along s–a–t, then two along s–b–t, giving value four. Residual path s–a–b–t permits one more. Source arcs now carry three and two, total five. Cut {s} has capacity $3+2=5$, so weak duality certifies maximum flow five and minimum cut five. No positive residual leaves s; its final reachable set is exactly {s}.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Optimization and matching solutions (continued)

6

Adding sorted edges without cycle checks can create a cycle, and adding every edge need not yield a forest. DSU tracks components: accept and union only when find(u)≠find(v). On a connected graph stop after n−1 accepted edges; on disconnected input produce one minimum spanning tree per component.

7

In A, B, C, D, E, F order, initialize (0,∞,∞,∞,∞,∞). After A: (0,3,1,∞,∞,∞),with B←A, C←A. After C:(0,2,1,8,∞,∞),with B←C, D←C. After B:(0,2,1,4,8,∞),with D←B, E←B. After D:(0,2,1,4,4,∞),with E←D. After E:no change. Finalized sets grow {A},{A, C},{A, C, B},{A, C, B, D},{A, C, B, D, E};F stays infinite and permits stopping.

Path A–C–B–D–E costs 1+1+2+0=4. Arc inequalities are

$$d_B = 2 \le 0 + 3,\ d_C = 1 = 0 + 1,\ d_B = 2 = 1 + 1,$$
$$d_D = 4 = 2 + 2,\ d_D = 4 \le 1 + 7,\ d_E = 4 = 4 + 0,\ d_E = 4 \le 2 + 6.$$

Each finite value has its predecessor path; F is unreachable by graph structure. A negative loop at F does not change A's reachable region or these distances. An implementation requiring every graph edge nonnegative may conservatively reject it, but that stronger input rule does not mean the unreachable cycle affects A's mathematical answer.

8

If a nontree edge e is lighter than a maximum-weight T-path edge f, adding e and deleting f strictly improves T, so T is not minimum. Equality gives a distinct equal-weight spanning tree, so T is not unique. Thus unique optimality requires strict inequality for every e.

Conversely each nontree e is uniquely heaviest on its fundamental cycle. The cycle property excludes e from every MST. All nontree edges are excluded, so every MST's n−1 edges must equal T's, proving uniqueness.

In the weighted quadrilateral, delete any one of the four unit cycle edges to obtain total three. A tree using a diagonal costs at least 2+1+1=4. Therefore the complete MST list is {AB, BC, CD},{BC, CD, DA},{CD, DA, AB},{DA, AB, BC}. None of the edges lies in every MST:each cycle edge is omitted once, and both diagonals are always omitted. Equal optimum value does not mean equal tree.

https://github.com/xinhi6716/computer-science-self-study

Optimization and matching solutions (continued)

9

Use original arc order (sa, sb, ab, at, bt, ba). After the first augmentation the vector is (1,0,1,0,1,0). Direction b→a has original ba's forward record with capacity 2−0=2 and ab's reverse record with capacity f(ab)=1. Their aggregated search capacity is three, but it is not one original arc's remaining capacity.

Using forward ba on the next path increases sb, ba, at to obtain (1,1,1,1,1,1). The ab/ba circulation has one unit each but net flow value two. Using reverse ab instead increases sb, at and decreases ab, giving (1,1,0,1,1,0),without that circulation. Both obey arc bounds and conservation, and cut {s} has capacity two, certifying both maximum. The value is unique; original-arc optimum flows need not be.

10

Take M={a1,b2,d3},size three. Search from unmatched c through nonmatching edges to 1,2,then matched edges to a, b. Their other neighbors remain 1,2. Thus Z_L={a, b, c},Z_R={1,2};no unmatched right vertex is reachable. Deficiency 3−2=1 proves at most 4−1=3 edges, attained by M.

The constructed cover is (L−Z_L)∪Z_R={d,1,2},size three, covering every edge. Cover {1,2,3} is another optimum; uniqueness is unnecessary. After adding c4,matching {a1,b2,c4,d3} is perfect of size four, the largest possible. No unmatched left starts remain, so both reachable sets are empty and cover L has size four. Four disjoint matched edges give a matching lower bound on every cover.

11

Necessity holds because all demand from S must enter N(S),whose total capacity is b(N(S)).

For sufficiency let D=Σd_i. Build s→i capacity d_i,allowed i→j capacity D+1,and j→t capacity b_j. Cut {s} has capacity D. Any cut crossing a left-to-right arc has capacity at least D+1 and cannot be smaller. A cut with no such crossing and source-side left set S must include N(S) on its source side. Its capacity is at least

$$D - d(S) + b(N(S)) \geq D.$$

Thus every cut has capacity at least D and max-flow min-cut gives flow D. Integral capacities admit an integral optimum. All source-left arcs saturate, meeting every demand. For D=0,the zero flow also satisfies the construction.

Optimization and matching solutions (continued)

In the case, send two units A→2 and one B→1. This meets demands and receiver capacities. With pair capacities one, this becomes infeasible: B must fill receiver one, leaving A only receiver two, but pair A2 carries at most one instead of required two. All neighborhood-total inequalities still hold, so they are no longer sufficient. Put unit capacities on pair arcs and use the full cut conditions for that model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization and matching solutions (continued)

Verification code appendix: residual_network.py

```
"""Edmonds–Karp with separate residual pairs for every original arc ID."""
from collections import deque

def maximum_flow(n: int, arcs: list[tuple[int,int,int]], source: int, sink: int):
    if not 0 <= source < n or not 0 <= sink < n or source == sink:
        raise ValueError("source and sink must be different valid vertices")
    graph=[[] for _ in range(n)]
    originals=[]
    for u,v,capacity in arcs:
        if not 0 <= u < n or not 0 <= v < n or not isinstance(capacity,int) or capacity < 0:
            raise ValueError("arc endpoints must be valid and capacity a nonnegative
integer")
        at_u=len(graph[u]);at_v=len(graph[v])+(u==v)
        graph[u].append([v,at_v,capacity])
        graph[v].append([u,at_u,0])
        originals.append((u,at_u,capacity))
    value=0
    while True:
        parent=[None]*n;parent[source]=(source,-1);queue=deque([source])
        while queue and parent[sink] is None:
            u=queue.popleft()
            for i,(v,reverse,residual) in enumerate(graph[u]):
                if residual>0 and parent[v] is None:
                    parent[v]=(u,i);queue.append(v)
        if parent[sink] is None:
            flows=[capacity-graph[u][i][2] for u,i,capacity in originals]
            reached={v for v in range(n) if parent[v] is not None}
            return value,flows,reached
        amount=None;v=sink
        while v!=source:
            u,i=parent[v]
            amount=graph[u][i][2] if amount is None else min(amount,graph[u][i][2])
            v=u
        v=sink
        while v!=source:
            u,i=parent[v];record=graph[u][i]
            record[2]-=amount
            graph[v][record[1]][2]+=amount
            v=u
        value+=amount

if __name__=='__main__':
    arcs=[(0,1,1),(0,2,1),(1,2,1),(1,3,1),(2,3,1),(2,1,2)]
    value,flows,source_side=maximum_flow(4,arcs,0,3)
    assert value==2 and source_side=={0}
    assert all(0<=flow<=arc[2] for flow,arc in zip(flows,arcs))

print({'value':value,'original_arc_flows':flows,'minimum_cut_source_side':sorted(source_side)
})
```

Optimization and matching solutions (continued)

Verification code appendix: verify_graphs.py

```
"""Check the chapter's shortest-path, MST, and max-flow examples."""

import heapq


def dijkstra(n: int, edges: list[tuple[int, int, int]], source: int, directed: bool = True) -
> list[float]:
    if not 0 <= source < n:
        raise ValueError("source must be a vertex")
    if any(w < 0 or not 0 <= u < n or not 0 <= v < n for u, v, w in edges):
        raise ValueError("this implementation requires all graph edges to have nonnegative
weights")
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
        if not directed:
            graph[v].append((u, w))
    dist = [float("inf")] * n
    dist[source] = 0
    queue = [(0, source)]
    while queue:
        d, u = heapq.heappop(queue)
        if d != dist[u]:
            continue
        for v, w in graph[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(queue, (nd, v))
    return dist


def mst(n: int, edges: list[tuple[int, int, int]]) -> int:
    parent = list(range(n))

    def find(x: int) -> int:
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    total = used = 0
    for w, u, v in sorted((w, u, v) for u, v, w in edges):
        ru, rv = find(u), find(v)
        if ru != rv:
            parent[ru] = rv
            total += w
            used += 1
    assert used == n - 1
    return total
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization and matching solutions (continued)

```
def max_flow(capacity: list[list[int]], source: int, sink: int) -> int:
    n = len(capacity)
    if source == sink or not 0 <= source < n or not 0 <= sink < n:
        raise ValueError("source and sink must be different vertices")
    if any(len(row) != n or any(c < 0 for c in row) for row in capacity):
        raise ValueError("capacity must be a square nonnegative matrix")
    residual = [row[:] for row in capacity]
    flow = 0
    while True:
        parent = [-1] * n
        parent[source] = source
        queue = [source]
        for u in queue:
            for v, cap in enumerate(residual[u]):
                if cap > 0 and parent[v] == -1:
                    parent[v] = u
                    queue.append(v)
                    if v == sink:
                        break
        if parent[sink] == -1:
            return flow
        add = float("inf")
        v = sink
        while v != source:
            u = parent[v]
            add = min(add, residual[u][v])
            v = u
        v = sink
        while v != source:
            u = parent[v]
            residual[u][v] -= add
            residual[v][u] += add
            v = u
        flow += add


assert dijkstra(2, [(0, 1, 2)], 0) == [0, 2]
assert dijkstra(2, [(0, 1, 2)], 1) == [float("inf"), 0]
assert dijkstra(2, [(0, 1, 2_000_000_000)], 0) == [0, 2_000_000_000]
assert mst(4, [(0, 1, 1), (1, 2, 2), (2, 3, 3), (0, 2, 4), (1, 3, 5)]) == 6

cap = [[0] * 4 for _ in range(4)]
cap[0][1], cap[0][2], cap[1][2], cap[1][3], cap[2][3] = 3, 2, 1, 2, 3
assert max_flow(cap, 0, 3) == 5

print("graph examples: PASS")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Optimization and matching solutions (continued)

References and attribution

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study